

막힘을 질문으로

Swift 초심자의 막힘을 구체적인 질문으로 바꾸는 앱을 만들며,
나 역시 막힘을 질문으로 바꾸는 법을 배웠다.

APP

막힘 → 언어화 → 질문

MY LEARNING

모호한 생각 → 질문 → 구조 → 확인

01 내가 만들고자 한 앱

Swift 초심자가 “모르겠어요”에서 멈추지 않고, 자신이 어디에서 왜 막혔는지를 구체적인 질문으로 바꾸도록 돕는다.

PROGRAMMING BEGINNER

컴퓨팅 사고력 학습 지원 앱

Swift 언어를 도구로 활용한

답을 바로 주는 대신,
학습자가 자신의 사고와 막힘을 꺼내 볼 수 있게 한다.

문제 구조화

사고 기록

최소한의 Swift 개념 체계

질문 생성

사용자의 변화

01

막힘 감지

“어디서부터 모르겠는지
잘 모르겠어요.”

→

02

아는 것 분리

내가 이해한 것과
확신 없는 것을 나눈다.

→

03

모르는 지점 언어화

막힌 위치·조건·시도를
말로 꺼내 본다.

→

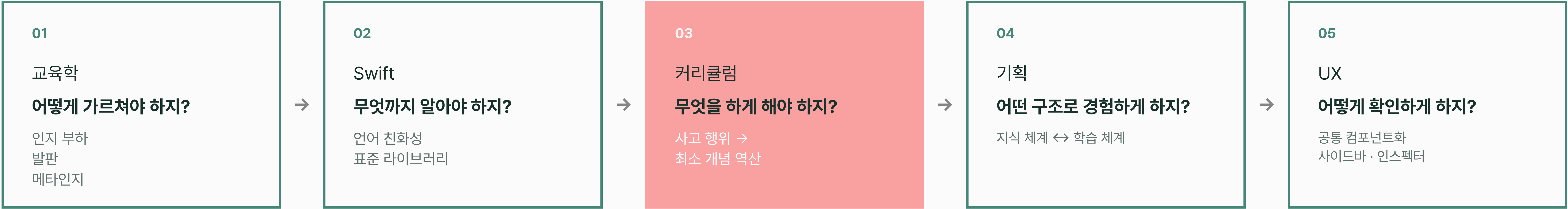
04

구체적인 질문

“나는 여기까지 이해했고,
이 지점이 왜 다른가요?”

02 내가 실제로 배운 경로

처음부터 정답을 알고 만든 것이 아니라, 한 번 막힐 때마다 다음 질문이 생겼다.



03 질문의 해상도가 높아진 순간

“무엇을 가르치지?” → “어떤 사고를 하게 하지?”

BEFORE

Swift를 도구로 활용해서
컴퓨팅 사고력을 키우기 위한
기초 커리큘럼을 만들어보자.

목표는 있었지만,
학습자의 사고가 아직 보이지 않았다

→

AFTER

“배울 문법”이 아니라
학습자가 할 수 있어야 하는
사고 행위로 먼저 정의하자.

그리고 그 사고에 필요한
최소 Swift 개념을 역산하자

Question 02

Swift를 쓸 거야

→ 왜 Swift여야 하지?

당연하게 둔 전제를 다시 질문했다.

Question 03

지식을 어떻게 정리하지?

→ 무엇이 필요하지?

사용자의 사고 흐름을 먼저 생각했다.

04 AI와 함께 하며 배운 것

AI는 정답을 대신 내는 도구보다, 내 생각을 밖으로 꺼내고 더 정확한 질문으로 바꾸는 도구에 가까웠다.

내게 지식과 경험이 적은 분야를 수행해야 할 때,
최소한의 사전 작업을 위한 시간에 필요한 전략

학습의 길잡이는 AI에 맡기되,
지식의 이해 방법과 정도는 내가 직접 판단하고,
내가 혼자서 다음에 필요할 때 지식을 이끌어낼 수 있는가를 통해 결과를 확인한다.

개발을 AI와 함께 할 때 내가 이전부터 채택했던 전략이며,
앞으로도 계속 중점적으로 함양해야 할 전략

생각은 내가 직접 하고, 구현은 AI에 맡기고,
구현 결과에서 내 의도가 담기지 않은 부분을 빠르게 찾아내고
그 의도가 왜 담기지 않았는지 생각해보는 경험을 쌓는다.

스태이지 1의 '챕터 1. 문제를 입력, 출력, 규칙으로 바라보기'의 학습 컨텐츠를 구성해볼까 필요한 컴포넌트를 유형화해보려고 해. 나중에 일에 저장할 자료구조를 고려하기는 하되, 지금은 모델 설계를 위함이라 일단 더 좋은 학습 컨텐츠를 만드는 것을 우선으로 생각할거야.

macOS 앱의 메인 도킹부 화면에서, 스태이지 1으로 들어거서 챕터 1으로 들어갔을 때, 화면에 있어야 할 것은 학습 컨텐츠, 사용자 동작 버튼, 코드 색션이 우선 생 각나. 내가 각각에 대한 내 생각을 적을 테니, 교육학적으로 사용자의 개념 지도, 인지 부하, 학습 발판 설계, 메타인지를 고려하여 컴퓨팅 사고력을 함양하기 위해 어떻게 구성해야 할지 함께 고민해보고 아이디어이션해보는 거야. 스태이지 1은 컴퓨팅 사고력을 얻기 위해 Swift를 도구로 사용할 준비를 마칠 수 있도록 돕는 것임을 이해하고, 비선형적인 Swift 문법 체계에서 토대를 잡을 일부 지식을 선형적으로 학습할 수 있도록 돕는 것임을 이해해야 함. 그러면서도 단순히 지식을 제공 하는 것뿐만 아니라 스스로 생각하는 연습을 조금씩 익숙해지게, 흥미를 가지고 몰입해서 학습할 수 있게 해야 하는 것도 이해해야 함.

1. 학습 컨텐츠: 화면을 가로로 나눠 절반 정도를 차지하기. 페이지 단위로 나눠서 인지 부하를 줄이기. 페이지 내에서 컨텐츠가 각 페이지에 필요한 컴포넌트를 배치하기. 컴포넌트는 제목, 일반 내용 같은 기본적인 것부터 주의사항, 토큰(누르면 추가 내용 펼쳐지고 접히는), 다이어그램 (행, 열, 그리드) 등 학습 컨텐츠에 들어갈 수 있는 컴포넌트를 유형화하고 각각의 디자인을 잡아서 내부 내용을 채우면 각 디자인에 따라 사용자 가 '아 이런 주의사항이구나' 를 점점 무의식적으로 알면서 컨텐츠를 받아들일 수 있도록 하기. 생각하는 힘을 기르기 위해 학습 컨텐츠에 어떤 조치가 필요할지 생각해보기.

2. 사용자 동작 버튼 4가지 도구를 눌러 활성화하기, 페이지 이동하기, 코드 색션 관련 버튼 등 사용자가 직접 실행하며 학습할 수 있도록 하기

3. 코드 색션 코드 색션이 어려운데, 초반에는 chunking을 연습할 겸 코드를 불박하고 그 하 나의 블록 내 구문부 각 단어의 이해를 각 탭터에 맞게 할 수 있도록 하기 간단히 <

Concriculum

정보를 깊게 타입으로 표현하기

CHAPTER 2 · 3 / 8

정보에 맞는 타입 선택하기

정보의 길모양이 아니라 의미, 허용할 상태, 이후 할 일을 근거로 타입을 선택한다.

기억 연결하기

1 String, Int, Double, Bool은 각각 어떤 일을 맡기 좋겠는?

기억할 방법

타입은 깊게 종류론 아니라 깊게 허용할 일을 정한다.

어떤 학습자의 경험

실제 문제에서는 타입 이름을 알아보는 데서 멈추지 않고 어떤 표현이 목적에 맞는 지 선택한다.

상황

같은 화면 문구, 다른 내부 값

주문 화면의 금액, 회원 여부, 발행권 모두 글자로 보이지만 앱은 계산-판단-소수-보존이라는 서로 다른 일을 해야 한다.

상황 이해

회원 20,000원 · 내부에서 더하고 본다.

회원 여부

회원 확인 · 할인 적용 여부를 판단한다.

발행 여부

회원 4.5 · 소수 부분을 보존한다.

먼저 생각해 보기

화면에 글자로 보인다는 이유로 세 정답을 모두 String으로 저장해도 될까?

비교하여 관찰하기

비교 기준

화면에 보이는 모양과 내부에서 필요한 일

"20,000원" ↔ 20_000

표시 글자와 계산할 정수의 목적이 다르다.

"회원" ↔ true

표시 글자와 두 경우 상태의 목적이 다르다.

"4.5" ↔ 4.5

표시 글자와 소수 수치의 목적이 다르다.

이전

3 / 8

다음

Xcode 프로젝트 상위 폴더에 docs 폴더를 생성하고 Manyfast MCP를 통해 가져온 최근 프로젝트의 PRD와 기능명세서를 활용해서 기획의 구조화를 구현으로 옮기기 위한 Codex 디렉트 md 파일을 작성해서, macOS 프로그램을 만들 준비를 시작해보자.

우선 아키텍처로 TCA를 활용하는 게 어떨까 싶는데, 나의 현재 상태는 MVVM + Service, Repository 형태를 잘 이해하고 있고 TCA는 이번엔 처음 써볼 생각을 가지고 있는 거라 이 기획과 구현에 알맞은 선택인지 판단해야 해. 또한 도메인 모델과 영속 저장 Repository 계층을 경계로 하게 될지, 다른 방법이 있는지, 어떤 계층과 경계를 가지게 될지 생각해. 그리고 계층별, 기능별, 일일할 폴더들 중 어떤 방식을 활용할지 선택해야 하므로 구현 순서와 방식 을 고려해서 내가 선택할 수 있도록 각각 어떤 방식으로 진행할 수 있는지를 생각해. 그리고 macOS에서는 라우팅을 어떤 방식으로 하는지 잘 모르겠어, 가능한 방식을 마련가지로 내 가 선택할 수 있도록 각각 어떤 방식으로 진행할 수 있는지를 생각해.

가장 처음으로 ③ Chapter 2 - 정보를 깊게 타입으로 표현하기 를 학습 컨텐츠로 만들거고 이전 Manyfast 기능명세서에서 사고 행위 기반 단계별 카이클럼 - 스테이지 1 일반 학습 활동의 챕터 2 의 제본 내용을 참고로 해. 그래서 컨텐츠도 학습 체계 Chapter 2 컨텐츠를 이어서 체계화 개념들과 연결되어 있어. 이것이 단순히 연결되는 것보다 지식 체계 속 개념들을 나에게 맞추어, 개념들 간의 연결도 나에게 맞추어 할 수 있도록 하는 것도 중요한 기능이야. 그 부분을 아가 설정하지 않 아서 전반적으로 추가되어야 해.

4. Chapter 2 관련 표현 방식에서 Markdown 지동 변환기는 그냥 내 md 파일을 화면화 하 기 위한 도구를 만든다고 이해하면 되나? 5. 지금 사이드바의 역할에 대해 더 고민해보고 싶어. 추천안은 NavigationSplitView이지만 나 는 사이드바가 스테이지 1 범위로 보여주는 것 상에 한하여 학습 화면에 통합할 수 없게 만든 도메인 될 수 있다고 생각해. 오히려 지식 체계를 보여주지, 모든 지식 체계를 보여주는 것 이 아니라 내 학습 체계에 맞는 지식 체계를 보여주지, 그것이 상세 영역의 학습 영역을 진행할 때 따라 내 언어로 표현한 부분이 사이드바에서도 업데이트되며 그 탭터가 어떻게 나에게 맞춤 형으로 변화하고 있는지 보여주는 장소로 활용하고 싶는데, 사이드바 구현에 이해가 없애 서 이것이 어떻게 가능할지 알려줬으면 좋겠어.

추천안에서 더 생각해볼 부분을 이야기할 테니까 나머지는 그대로 진행하는 게 좋다는 이야기.

이 범위의 선행은 선행한 ③ 학습 체계 를 지나오면서 비선형적인 ③ 지식 체계 를 사용자가 배운 것들을 기반으로 제시하는 개념 언어를 나만의 언어로 바꿔가며 나만의 지식 체계를 가질 수 있 게 배우는 것을 목표로 해. 그래서 컨텐츠도 학습 체계 Chapter 2 컨텐츠를 이어서 체계화 개념들과 연결되어 있어. 이것이 단순히 연결되는 것보다 지식 체계 속 개념들을 나에게 맞추어, 개념들 간의 연결도 나에게 맞추어 할 수 있도록 하는 것도 중요한 기능이야. 그 부분을 아가 설정하지 않 아서 전반적으로 추가되어야 해.